Partner task pack

8 prompts for Gemini 4 Argon - the operator runs them

8 TASKS · BRIDGE 5.0.0

FOR	Gemini 4 Argon (model, not an agent)
FROM	Arena coding agent, branch arena/706ae6d5-tunneling
REPO	Clovetie/Tunneling
GENERATED	2026-11-03
HOW TO USE	Paste one prompt + the attached files into a session

---

INSIDE

How to use this pack · Task index · Ground rules · Operator coordination · G1-G8 task cards (objective, definition of done, verification, verbatim prompt, report-back template) · Appendix: seeded bus tasks

How to use this pack

Gemini 4 Argon is a **model**, not an autonomous agent: it cannot run commands, open a repo, claim a bus task, or push a branch. So each task below is a **self-contained prompt** plus the files it needs. The human operator (or an agent wrapper) does the running: attach the listed files, paste the prompt, take the returned patch/docs, run the verification commands, and record the result on the Agent Bus.

- **One task per session.** Attach only the files listed under 'Attach' to keep context focused.
- **Paste the prompt block verbatim.** It already names files, constraints and the exact deliverables.
- **Reject invented measurements.** G2 and G7/G8 require real numbers from a machine that can reach Studio; placeholders must stay visible.
- **Apply and verify.** Run the verification commands and `python3 -B roblox-bridge/test_bridge_mcp.py`; the suite must stay green (25 tests).
- **Record it.** Ask the operator to claim the task on the bus, then complete it with artifacts (files touched, test output).

Task index

ID	Task	Priority	Owner files
G1	Named-tunnel and custom-domain runbook	High	`roblox-bridge/tunnel_manager.py, docs/OPS.md (new section only)`
G2	Public-tunnel latency budget (p50/p95)	High	`docs/OPS.md, jobs/ (measurement script)`
G3	Ops tab responsive layout and SSE reconnect	Normal	`roblox-bridge/server.py (dashboard HTML/JS only)`
G4	Multi-agent MCP prompt recipes	Normal	`roblox-bridge/mcp_core.py (PROMPTS and get_mcp_prompt only)`
G5	Stale-agent sweeper for the task board	Normal	`roblox-bridge/bus_core.py, roblox-bridge/test_bridge_mcp.py`
G6	Incremental handoff brief (since / summaryOnly)	Low	`roblox-bridge/bus_core.py, tests`
G7	Live Studio acceptance run (real peer + real tunnel)	High	`docs/OPS.md section 10 only`
G8	Kill-switch drill evidence	Normal	`docs/OPS.md section 10 only`

What each card contains

- **Owner files** - the only files the patch may touch. Anything else is out of scope for that task.
- **Context** - the background the model cannot infer from the attachment alone.
- **Objective** - one paragraph, no acceptance criteria hidden inside it.
- **Definition of done** - the checklist the operator applies before accepting a patch.
- **Verification** - the exact commands that must pass, with the expected result.
- **Paste-ready prompt** - the block to paste verbatim, attachments attached and nothing else added.
- **Report back** - the template the operator fills in and posts to the Agent Bus.

Ground rules for every task

- Python 3 standard library only. The bridge must stay pip-free at runtime (reportlab/pygments are used only to build these PDFs).
- Never weaken auth. `bridge.token` is the admin owner credential; scoped tokens are additive.
- Never let the bridge publish itself without a token; the tunnel guard must keep returning state='blocked'.
- Runtime state belongs in gitignored `roblox-bridge/bridge_state/`; never commit tokens, audit logs or bus data.
- Keep the JSON shapes stable: `/api/health`, `/api/tunnel`, `/api/ops`, `/api/metrics`, `/api/bus`, `/api/audit`.
- Do not rename existing MCP tools or bus actions; additive changes only.
- Every change keeps `python3 -B roblox-bridge/test_bridge_mcp.py` green (25 tests).
- Report honestly: if a step needs a live Roblox Studio peer, say it is unverified rather than claiming success.

Operator coordination (Agent Bus)

The model cannot claim tasks, so the operator does it before pasting the prompt, and completes it afterwards with the artifacts. Commands:

```
python roblox-bridge/arena_studio.py bus tasks
python roblox-bridge/arena_studio.py bus task-claim  --task task-xxxxxxxx --agent agent-xxxxxxxx
python roblox-bridge/arena_studio.py bus task-complete --task task-xxxxxxxx --agent agent-xxxxxxxx \
    --result "G1 docs + test landed" --detail "see tunnel_manager.py diff"
python roblox-bridge/arena_studio.py bus post --agent agent-xxxxxxxx \
    --to "Claude" --subject "G1 review" --body "patch attached, tests green"
```

Seeding the whole work order into a fresh bridge: `python roblox-bridge/tools/seed_agent_bus.py` (reads `handoff/bus_seed.json`).

If the output cannot be applied

- Return the task to the model with the failing command and the first 40 lines of output; do not let it guess.
- If two attempts fail on the same file, split the task or ask for the design decision only, with no patch.
- If a task turns out to need live Studio access (G7, G8), stop and hand it back to the operator instead of simulating results.
- Never mark a bus task done on the model's word; the verification commands decide.

G1 Named-tunnel and custom-domain runbook

High

Owner files	`roblox-bridge/tunnel_manager.py, docs/OPS.md (new section only)`
Attach to prompt	tunnel_manager.py, docs/OPS.md
Out of scope	server.py, mcp_core.py, auth_core.py, bus_core.py, telemetry.py, existing tests

Context

The named-tunnel path exists but has only ever been exercised with a fake binary. Nothing in the repo proves the composed command is correct for Cloudflare's named-tunnel mode, and the operator docs stop at quick tunnels.

Objective

Make the named Cloudflare tunnel path production-ready and document it for a non-expert. The bridge already composes `cloudflared tunnel --no-autoupdate run --token <T>` for named tunnels and `--url <local>` for quick tunnels. Harden the named path and write the runbook.

Definition of done

- Malformed/empty token fails fast with an actionable message (state `error`, no silent loop).
- `--tunnel-name` is validated or removed from the CLI surface if it has no effect.
- `docs/OPS.md` gains a step-by-step section: create tunnel, DNS/CNAME, token, run bridge, verify `/api/tunnel` shows a stable URL.
- A unit test asserts the composed command contains `run --token` and never `--url` for named tunnels.

Verification

```
python3 -B roblox-bridge/test_bridge_mcp.py TestV5OpsPlatform
python3 roblox-bridge/arena_studio.py tunnel status
```

Paste-ready prompt

```
You are working on the Arena Roblox Studio bridge, version 5.0.0, file roblox-bridge/tunnel_manager.py.
Context: TunnelManager supervises cloudflared/ngrok. _command() returns:
  named: [binary, 'tunnel', '--no-autoupdate', 'run', '--token', token]
  quick: [binary, 'tunnel', '--url', 'http://127.0.0.1:<port>', '--no-autoupdate']
  ngrok: [binary, 'http', str(port), '--log', 'stdout', '--log-format', 'logfmt']
Constraints: Python 3 stdlib only; never publish a token-less bridge; keep the state machine
(stopped/starting/online/degraded/reconnecting/unavailable/error) and the /api/tunnel JSON shape.
Do these three things:
1. Validate the named token: reject empty/whitespace and obviously malformed values before spawn, set
state='error' with an actionable lastError, and log one line to logTail.
2. Decide whether --tunnel-name should be used (Cloudflare ingress configures the hostname) or removed;
if kept, expose it only as display metadata, never as a CLI flag that changes routing.
3. Write the runbook section for docs/OPS.md: 'Named tunnel with a stable hostname'. Include Cloudflare
Zero Trust steps (create tunnel, public hostname -> http://127.0.0.1:8077, copy token), the exact
commands (python setup.py --tunnel --tunnel-token $TOKEN), verification commands, and three
troubleshooting cases (401 on /mcp, trycloudflare URL still appearing, state=unavailable).
Deliver: unified diff of tunnel_manager.py, the markdown section, and a unittest method that asserts the
named command contains 'run --token' and not '--url'.
Do not modify server.py, mcp_core.py, auth_core.py, bus_core.py, telemetry.py, or existing tests.
```

Report back

```
Result      : pass / pass-with-caveats / blocked
Files       : <paths touched by the applied patch>
Tests       : python3 -B roblox-bridge/test_bridge_mcp.py -> Ran 25 tests ... OK
Evidence    : <diff / JSON / doc section>
Unverified  : <anything that needs live Studio or a real tunnel>
Open risks  :
```

G2 Public-tunnel latency budget (p50/p95)

High

Owner files	`docs/OPS.md, jobs/ (measurement script)`
Attach to prompt	arena_studio.py, docs/OPS.md
Out of scope	bridge source (this task is tooling + docs only)

Context

Every support question about the tunnel comes down to latency, and the project has no numbers. The sandbox that wrote this bridge cannot reach trycloudflare.com, so the numbers must come from the operator's machine; the script and the table are still the model's job.

Objective

Quantify the tunnel tax so users can decide whether a named tunnel or a local agent is worth it. Measure `/api/jobs` round trips for three Studio tools, localhost vs the public URL, 20 samples each.

Definition of done

- A small script (jobs/measure_tunnel_latency.py, stdlib only) that takes --url, --token, --runs and prints p50/p95/max.
- Runs against a live Studio peer; records JSON output alongside the numbers.
- `docs/OPS.md` gains a 'Tunnel latency budget' table: tool, local p50/p95, tunnel p50/p95, delta.
- States the sample size, date, provider/flavor and region honestly; no invented numbers.

Verification

```
python3 jobs/measure_tunnel_latency.py --runs 20 --url http://127.0.0.1:8077
python3 roblox-bridge/arena_studio.py metrics --prometheus
```

Paste-ready prompt

```
Write a stdlib-only measurement script for the Arena bridge at jobs/measure_tunnel_latency.py.
It must POST to <url>/api/jobs with {'type': <tool>, 'payload': <args>, 'wait': 60, 'dm': 'edit'} and
header 'Authorization: Bearer <token>' (or token query param), then time the round trip. Tools to
measure: ping_studio {}, get_studio_state {}, script_read {'path': 'ServerScriptService'}.
CLI: --url, --token, --runs (default 20), --tools, --json-out. Print per-tool count/p50/p95/max and a
markdown table. Handle 401 and no-peer (status=error reason=no_peer) by failing with a clear message.
Then give me the markdown section 'Tunnel latency budget' for docs/OPS.md, with placeholders only if
measurements are unavailable, and an explicit note that numbers must be produced on the operator's
machine because sandbox egress cannot always reach trycloudflare.com.
Do not modify the bridge source; this is tooling plus documentation.
```

Report back

```
Result      : pass / pass-with-caveats / blocked
Files       : <paths touched by the applied patch>
Tests       : python3 -B roblox-bridge/test_bridge_mcp.py -> Ran 25 tests ... OK
Evidence    : <diff / JSON / doc section>
Unverified  : <anything that needs live Studio or a real tunnel>
Open risks  :
```

G3 Ops tab responsive layout and SSE reconnect

Normal

Owner files	`roblox-bridge/server.py (dashboard HTML/JS only)`
Attach to prompt	server.py (dashboard_html function), OPS.md section 5
Out of scope	any Python in server.py outside dashboard_html()

Context

The Ops tab is one large HTML string inside server.py. It was built desktop-first, and the EventSource has no reconnect path, so a bridge restart silently freezes the live feed.

Objective

The Ops tab works on desktop but crowds below 420 px, and the EventSource dies permanently on a server restart. Make it mobile-usable and self-healing.

Definition of done

- At 390x844 the six stat cards wrap to two per row; the task board and tunnel panels stack; no horizontal scroll.
- EventSource reconnects with exponential backoff (1s -> 30s) and the feed shows 'reconnecting' vs 'closed'.
- New button: 'Copy MCP config (tunnel)' producing a JSON snippet with the public /mcp URL and the current token.
- No Python changes in server.py outside dashboard_html(); node --check on the extracted script passes.

Verification

```
python3 -c "import sys;sys.path.insert(0,'roblox-bridge');import
server;print(len(server.dashboard_html()))"
node --check /tmp/dash.js # after extracting the <script> block
```

Paste-ready prompt

```
You are editing the single-file dashboard returned by dashboard_html() in roblox-bridge/server.py
(Python raw string containing HTML/CSS/JS). Features: tabs, tick() polling /api/state every 1.5s, an Ops
tab with tunnel controls, agent bus, metrics and an EventSource on /api/events.
Return only the changed HTML/CSS/JS fragments, with enough surrounding context to apply them:
1. CSS: at max-width 720px make .grid two columns, stack .two-col, reduce #tab-ops card padding, and
prevent long tunnel URLs from forcing horizontal scroll (word-break: break-all; overflow-wrap:anywhere).
2. JS: replace the single EventSource handler with connectEvents(delay) that reconnects with
Math.min(delay*2, 30000) after onerror, updates #ev-btn to 'Reconnecting (Ns)' and resets on success.
Never leak duplicate sources; closing must cancel timers.
3. Add a 'Copy MCP config (tunnel)' button in the tunnel card. It copies
{"mcpServers":{"roblox-studio":{"url":"<publicUrl>/mcp","headers":{"Authorization":"Bearer <token>"}}}}
using the current token input and the publicUrl from the last /api/ops response.
Keep vanilla JS, no frameworks, no external assets. Ensure the extracted <script> passes node --check.
```

Report back

```
Result      : pass / pass-with-caveats / blocked
Files       : <paths touched by the applied patch>
Tests       : python3 -B roblox-bridge/test_bridge_mcp.py -> Ran 25 tests ... OK
Evidence    : <diff / JSON / doc section>
Unverified  : <anything that needs live Studio or a real tunnel>
Open risks  :
```

G4 Multi-agent MCP prompt recipes

Normal

Owner files	`roblox-bridge/mcp_core.py (PROMPTS and get_mcp_prompt only)`
Attach to prompt	mcp_core.py (PROMPTS, get_mcp_prompt), bus_core.py action list
Out of scope	bus_core.py, server.py, existing prompts and MCP tools

Context

The bus works, but agents arrive without a protocol: they claim nothing, post nothing, and leave work half-done. MCP prompts are the cheapest place to teach the protocol because they render inside the client.

Objective

Agents do not use the bus well without instructions. Add three workflow prompts that render live bus state and teach handoff, standup and triage.

Definition of done

- prompts/list returns 7 prompts: the existing 4 plus multi_agent_handoff, bus_standup, task_triage.
- Each prompt embeds the live roster/tasks when a bridge is reachable, and a safe fallback sentence otherwise.
- Prompt text tells the model to claim before editing, post results, and complete with artifacts.
- tests: prompts/list count and one prompts/get per new prompt with live bus state.

Verification

```
python3 -B roblox-bridge/test_bridge_mcp.py
python3 roblox-bridge/arena_studio.py bus tasks
```

Paste-ready prompt

```
In roblox-bridge/mcp_core.py there are 4 MCP prompts rendered by get_mcp_prompt(name, arguments). The
bridge now has an agent bus (bus_core.AgentBus) with actions: register, agents, inbox, post, tasks,
task_create, task_claim, task_complete, export. The MCP tool is agent_bus and the client exposes
client.bus_action({...}).
Add three prompts and their renderers:
- multi_agent_handoff: instruct an agent to export the board (agent_bus action=export), summarise
in-progress leases and blockers, and post a handoff message with the next concrete step.
- bus_standup: ask for what changed since a cursor, who owns what, and what is blocked; produce a 5-line
standup.
- task_triage: rank open tasks by priority and dependency depth, propose claim order, flag tasks with no
acceptance criteria.
Render live state by calling client.bus_action for stats/agents/tasks inside the renderer, wrapped in
try/except with a short fallback. Return: the PROMPTS entries, the get_mcp_prompt branches, and a
unittest asserting prompts/list == 7 and each new prompt returns non-empty text. Change nothing else.
```

Report back

```
Result      : pass / pass-with-caveats / blocked
Files       : <paths touched by the applied patch>
Tests       : python3 -B roblox-bridge/test_bridge_mcp.py -> Ran 25 tests ... OK
Evidence    : <diff / JSON / doc section>
Unverified  : <anything that needs live Studio or a real tunnel>
Open risks  :
```

G5 Stale-agent sweeper for the task board

Normal

Owner files	`roblox-bridge/bus_core.py, roblox-bridge/test_bridge_mcp.py`
Attach to prompt	bus_core.py (leases/presence), telemetry.py (metrics)
Out of scope	handle() action names, lease rules for agents that are still online

Context

Leases expire after 900 s, so an agent that dies keeps its task locked for a quarter of an hour and the board looks busy while nothing moves. A sweeper closes that gap without weakening leases.

Objective

A crashed or closed agent keeps its in_progress task until the lease expires, which can be 15 minutes. Release tasks held by agents that have been offline for a long time and tell the team.

Definition of done

- AgentBus.sweep_stale(offline_seconds=900) releases tasks owned by offline agents, sets status todo, clears the lease, records history and emits one broadcast alert per sweep.
- A counter-style stat (agentsReaped) is exposed through stats() and can be mirrored to bridge_agents_reaped_total by the caller.
- Does not touch leases whose owner is still online; idempotent when called twice.
- Unit test with injected timestamps proves release, alert, and no-op when online.

Verification

```
python3 -B roblox-bridge/test_bridge_mcp.py TestV5OpsPlatform
```

Paste-ready prompt

```
In roblox-bridge/bus_core.py, AgentBus stores agents with lastSeen/ttl and tasks with
lease={owner,claimedAt,expiresAt,seconds}. Implement sweep_stale(offline_seconds=900, now=None) that:
1. Finds agents whose lastSeen is older than offline_seconds or whose online flag is False.
2. Releases every task with status in_progress whose lease.owner (or assignee) is a stale agent:
status='todo', assignee=None, lease=None, history entry {action:'reaped'}, leaseExpiries+1.
3. Appends ONE broadcast message per sweep of kind 'alert', subject 'Stale agent tasks released',
listing task ids and the agent names.
4. Increments a counter exposed via stats() as agentsReaped and lastSweepAt.
5. Is safe to call twice (no duplicate alerts, no double release).
Use the existing lock and _save(); do not change handle() action names. Add AgentBus tests using an
injectable now or by mutating lastSeen. Return the full method, the stats() diff, and the test method.
```

Report back

```
Result      : pass / pass-with-caveats / blocked
Files       : <paths touched by the applied patch>
Tests       : python3 -B roblox-bridge/test_bridge_mcp.py -> Ran 25 tests ... OK
Evidence    : <diff / JSON / doc section>
Unverified  : <anything that needs live Studio or a real tunnel>
Open risks  :
```

G6 Incremental handoff brief (since / summaryOnly)

Low

Owner files	`roblox-bridge/bus_core.py, tests`
Attach to prompt	bus_core.py export_handoff
Out of scope	the current export signature/defaults, other bus actions

Context

Handoff briefs grow with the session, so by the twentieth message they are longer than the work they describe. Cursor-based export keeps them usable.

Objective

Handoff briefs currently contain everything. Let them be incremental so long sessions stay readable.

Definition of done

- export_handoff(path=None, include_done=True, since=0, summary_only=False) filters messages and events by cursor and omits full task detail when summary_only.
- handle(action='export') passes since/summaryOnly through.
- Existing export test stays green; new test proves only later messages appear.

Verification

```
python3 -B roblox-bridge/test_bridge_mcp.py TestV5OpsPlatform
```

Paste-ready prompt

```
In roblox-bridge/bus_core.py, export_handoff renders a markdown brief from agents, tasks, messages and
stats. Extend it with since: int = 0 and summary_only: bool = False.
Behaviour: messages with seq <= since are omitted; the task board shows only tasks updated after since
when since > 0, plus every currently open task regardless of age. summary_only renders one line per task
(status, priority, owner, title) with no detail/result/artifacts and drops the messages section.
Keep the returned dict {markdown, path, bytes}. Update handle() to pass p.get('since') and
p.get('summaryOnly'). Return the full function replacement and a unittest that posts a message, captures
the cursor, posts another, and asserts only the second appears with since=cursor.
```

Report back

```
Result      : pass / pass-with-caveats / blocked
Files       : <paths touched by the applied patch>
Tests       : python3 -B roblox-bridge/test_bridge_mcp.py -> Ran 25 tests ... OK
Evidence    : <diff / JSON / doc section>
Unverified  : <anything that needs live Studio or a real tunnel>
Open risks  :
```

G7 Live Studio acceptance run (real peer + real tunnel)

High

Owner files	`docs/OPS.md section 10 only`
Attach to prompt	none - needs the operator's PC
Out of scope	bridge source (docs section 10 only)

Context

This is the only task that cannot be finished from a text model's output alone: it needs a real Studio session and a real public URL. The model's deliverable is the script and the expected outputs; the operator's deliverable is the pasted evidence.

Objective

Prove the whole path with a real Roblox Studio place and a real public tunnel, then record the evidence. This is the gap the unit tests cannot close.

Definition of done

- `python setup.py --tunnel` prints the public URL; /api/tunnel shows state=online and health.ok=true.
- From an MCP client through the tunnel URL, bridge_ops status returns ok with studio_connected=true.
- self_test / MCP self_test reports studioScratch=pass.
- The JSON evidence (health, tunnel, self_test summary) is pasted under 'Live acceptance' in docs/OPS.md.

Verification

```
python3 setup.py --tunnel
python3 roblox-bridge/arena_studio.py ops
python3 roblox-bridge/arena_studio.py tunnel status
python3 roblox-bridge/arena_studio.py audit --verify
```

Paste-ready prompt

```
Here is the operator checklist for a live acceptance run of the Arena bridge 5.0.0. Turn it into a
precise, copy-pasteable acceptance script with expected outputs and failure triage.
Steps: (1) Studio open on a scratch place with the ArenaBridge.lua plugin enabled; (2) python setup.py
--tunnel; (3) confirm /api/tunnel state=online, publicUrl non-empty, health.ok=true; (4) from an MCP
client configured with that publicUrl and the bridge token call bridge_ops {action:'status'} and
agent_bus {action:'register'}; (5) run self_test with includeStudioSmoke=true; (6) verify audit --verify
ok; (7) stop Studio mid-job and confirm the reaper requeues (bridge_jobs_reaped_total increments, job
returns to queued).
For each step give: command, expected JSON fields, and what to do if it fails (401 -> token, no_peer ->
plugin not polling, unavailable -> no cloudflared/ngrok, blocked -> no token). Finish with a markdown
'Live acceptance' template for docs/OPS.md that includes date, Studio version, place id, provider, URL,
and pasted JSON.
```

Report back

```
Result      : pass / pass-with-caveats / blocked
Files       : <paths touched by the applied patch>
Tests       : python3 -B roblox-bridge/test_bridge_mcp.py -> Ran 25 tests ... OK
Evidence    : <diff / JSON / doc section>
Unverified  : <anything that needs live Studio or a real tunnel>
Open risks  :
```

G8 Kill-switch drill evidence

Normal

Owner files	`docs/OPS.md section 10 only`
Attach to prompt	none - needs the operator's PC
Out of scope	bridge source (docs section 10 only)

Context

The kill switch is the feature that makes the tunnel safe to leave on. It has unit coverage for the guard but no documented drill, so nobody knows what a broken guard looks like in practice.

Objective

Document what happens when Studio disappears mid-job, including the event feed and metrics before/after.

Definition of done

- A job submitted with dm=edit is running when Studio is closed.
- Within ~25 s the job returns to queued and bridge_jobs_reaped_total increments.
- After reopening Studio the job completes and the event sequence (job.enqueue, job.claim, job.requeued_orphan, job.claim, job.finish) is pasted as evidence.

Verification

```
python arena_studio.py metrics --prometheus | findstr bridge_jobs_reaped_total
python arena_studio.py audit --limit 20
```

Paste-ready prompt

```
Design a reproducible kill-switch drill for the Arena bridge 5.0.0 (reaper in server.py: jobs claimed by
a peer that stops polling for >20s are requeued; jobs exceeding their timeout are failed).
Give me: (1) an exact command sequence to start a long-running edit job, capture metrics before/after,
and close Studio at the right moment; (2) the expected event sequence in /api/events; (3) the exact
assertions (counter delta >= 1, job status transitions queued -> running -> queued -> running -> done);
(4) a markdown evidence template for docs/OPS.md with placeholders for the pasted JSON; (5) three
failure modes and how to distinguish them (job stuck running, job failed permanently, peer never
returns). Keep it runnable with arena_studio.py and curl/PowerShell only.
```

Report back

```
Result      : pass / pass-with-caveats / blocked
Files       : <paths touched by the applied patch>
Tests       : python3 -B roblox-bridge/test_bridge_mcp.py -> Ran 25 tests ... OK
Evidence    : <diff / JSON / doc section>
Unverified  : <anything that needs live Studio or a real tunnel>
Open risks  :
```

Appendix - seeded bus tasks

These are the board entries written by `tools/seed_agent_bus.py` from `handoff/bus_seed.json`. The JSON file remains the machine-readable source; the tables below are the same content in reviewable form.

Agents

Name	Role	Capabilities
Claude	builder	server-bridge, mcp, python, luau
Gemini 4 Argon	partner	roblox, qa, docs

Tasks

ID	Pri	Title	Detail
G1	High	G1 - Named-tunnel + custom-domain runbook	Test `cloudflared tunnel run --token` with a Zero Trust named tunnel, document DNS/CNAME setup, and make a malformed token fail loudly. Add a unit test for the composed command line. Owner files: roblox-bridge/tunnel_manager.py, docs/OPS.md.
G2	High	G2 - Tunnel latency budget (p50/p95)	Measure /mcp + /api/jobs overhead through the quick tunnel vs localhost for ping_studio, get_studio_state and script_read (20 calls each). Add the table to docs/OPS.md using metrics --prometheus.
G3	Normal	G3 - Ops tab responsive + SSE reconnect	Make the Ops tab work at 390x844: stack cards, add a 'copy MCP config with tunnel URL' button, and reconnect the EventSource with backoff. Touch only the dashboard JS/HTML inside roblox-bridge/server.py.
G4	Normal	G4 - Multi-agent MCP prompt recipes	Add prompts multi_agent_handoff, bus_standup and task_triage that teach agents to use the bus, rendering live bus state. Edit only PROMPTS and get_mcp_prompt in mcp_core.py.
G5	Normal	G5 - Stale-agent sweeper	Release tasks held by agents offline > 15 min, broadcast an alert, and increment bridge_agents_reaped_total. Unit-test with fake clocks; keep handle() action names stable.
G6	Low	G6 - Handoff brief `since` + summaryOnly	Let action='export' accept since (cursor) and summaryOnly so the brief contains only recent activity. Keep existing export tests green and add coverage.
G7	High	G7 - Live Studio acceptance run	With Studio open: setup.py --tunnel, start tunnel from the Ops tab, verify state=online and health.ok=true, call bridge_ops status from an MCP client, confirm self_test studioSmoke=pass. Paste JSON into docs/OPS.md 'Live acceptance'.
G8	Normal	G8 - Kill-switch drill	Stop Studio mid-job. Confirm bridge_jobs_reaped_total increments, the job returns to queued and completes when Studio reconnects. Record the event feed excerpt.